
Kat: declaring use of AI for purposes of animating a figure.

Me: The code I have here produces concentration profiles using static plots at particular time steps. I want to see a continuous evolution of the tracer field by imposing an animation to show the full progression of advection and diffusion processes over time. I believe IPython.display allows for rendering the animation to a notebook and FuncAnimation creates the animation itself. Can you impose this behaviour? I want to define a function that initiates a plot outline and another function that populates the plot with a timestep that updates with the text ‘Time = time array (s)’ for each step. Also, I want to save it out using ani.save

```
import numpy as np
import matplotlib.pyplot as plt

# spatial and time arrays
x_phys = np.linspace(0, L, n_grid)
time_arr = np.arange(c_ad.shape[0]) * dt

# remove ghost cells
c_ad_phys = c_ad[:, 2:n_grid+2]
c_df_phys = c_df[:, 2:n_grid+2]
c_an_ad_phys = c_an_ad[:, 2:n_grid+2]
c_an_df_phys = c_an_df[:, 2:n_grid+2]

# choosing time steps to plot
frames_to_plot = [0, int(len(time_arr)/3), int(2*len(time_arr)/3), -1]

ymin = np.min([c_ad_phys, c_df_phys, c_an_ad_phys, c_an_df_phys])
ymax = np.max([c_ad_phys, c_df_phys, c_an_ad_phys, c_an_df_phys]) * 1.1

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# advection plot on the left
ax1 = axes[0]
for f in frames_to_plot:
    ax1.plot(x_phys, c_ad_phys[f, :], linewidth=2,
             label=f'Num t={time_arr[f]:.1f}s')
    ax1.plot(x_phys, c_an_ad_phys[f, :], 'k--', linewidth=2,
             label=f'An t={time_arr[f]:.1f}s')

ax1.set_xlim(0, L)
ax1.set_ylim(ymin, ymax)
ax1.set_xlabel('x [m]', fontsize=18)
```

```
ax1.set_ylabel('c [g/L]', fontsize=18)
ax1.set_title('Advection', fontsize=18)
ax1.grid(True)
ax1.legend(fontsize=12)

# diffusion plot on the right
ax2 = axes[1]
for f in frames_to_plot:
    ax2.plot(x_phys, c_df_phys[f, :], linewidth=2,
             label=f'Num t={time_arr[f]:.1f}s')
    ax2.plot(x_phys, c_an_df_phys[f, :], 'k--', linewidth=2,
             label=f'An t={time_arr[f]:.1f}s')

ax2.set_xlim(0, L)
ax2.set_ylim(ymin, ymax)
ax2.set_xlabel('x [m]', fontsize=18)
ax2.set_ylabel('c [g/L]', fontsize=18)
ax2.set_title('Diffusion', fontsize=18)
ax2.grid(True)
ax2.legend(fontsize=12)

# increasing font size because it was too small
for ax in axes:
    ax.tick_params(axis='both', which='major', labelsize=16)

plt.tight_layout()
plt.show()
```

CHAT: Yes — what you're asking for is exactly the correct way to convert your current static snapshot comparison plot into a time-evolving animation.

Below is your code with the animation behaviour imposed using:

- **`matplotlib.animation.FuncAnimation`**
- **`init()` → sets up empty plot structure**
- **`update(frame)` → updates each timestep**

```
# ----------------------------
# INIT FUNCTION
# ----------------------------
def init():
line_ad_num.set_data([], [])
line_ad_an.set_data([], [])
```

```
line_df_num.set_data([], [])
line_df_an.set_data([], [])
time_text.set_text('')
return line_ad_num, line_ad_an, line_df_num, line_df_an, time_text


# ----------------------------
# UPDATE FUNCTION
# ----------------------------
def update(frame):

# advection
line_ad_num.set_data(x_phys, c_ad_phys[frame, :])
line_ad_an.set_data(x_phys, c_an_ad_phys[frame, :])

# diffusion
line_df_num.set_data(x_phys, c_df_phys[frame, :])
line_df_an.set_data(x_phys, c_an_df_phys[frame, :])

# time display
time_text.set_text(f'Time = {time_arr[frame]:.2f} s')

return line_ad_num, line_ad_an, line_df_num, line_df_an, time_text


# ----------------------------
# ANIMATION
# ----------------------------
ani = animation.FuncAnimation(
fig,
update,
frames=len(time_arr),
init_func=init,
interval=100,
blit=False
)
```

And using `ani.save()` → export as a GIF

```
display(HTML(ani.to_jshtml()))

ani.save('advection_diffusion_lines.gif', writer='pillow', fps=5)
```

Junqi: Caption polishing.

Original caption: Fig: Sensitivity experiments of baseline, high diffusivity coefficient and high flow velocity cases in 2D numerical advection-diffusion simulations. White arrows indicate the flow velocity, blue-yellow colors denote nitrate concentration and red contours represent the concentration of 10−4 g/L.

AI Reply: Figure X. Spatiotemporal evolution of nitrate concentration in 2D advection-diffusion simulations. The three rows compare the baseline scenario (top), a high diffusion case (middle), and a high flow velocity case (bottom) across four time steps (t=99, 249, 499, and 999 s). The background color map represents the nitrate concentration [g/L] on a logarithmic scale, with the red contour delineating the 10−4 g/L threshold. White arrows indicate the direction and magnitude of the uniform background flow velocity.

Haechan: converted text to LaTeX version

https://claude.ai/share/5eed7bdb-3b23-48d1-9004-d3880d958b08 (my prompt)

Original text: In stage 2, we will conduct sensitivity experiments using our 2-dimensional numerical model. We will evaluate the model's performance by adjusting current velocity and diffusivity based on a baseline experiment. In addition, by applying a new flow field and nitrate modeling, we aim to gain insight into how nitrate moves in the real world.

The parameters of the sensitivity experiments are given by the following table:

For the baseline scenario, we will set a constant current velocity of 0.01 m/s, a diffusivity of 0.01 m^2/s, and the initial nitrate concentration as a Gaussian function. To investigate how advection and diffusion respond to flow velocity and diffusivity, we will use a current velocity of 1.0 m/s for the high velocity scenario and a diffusivity of 0.02 m^2/s for the high diffusion scenario.

For the flow field, we plan to implement a classic boundary layer flow, where the current velocity u(y) follows a logarithmic profile relative to the distance from the shore (y) (Schlichting et al., 2016):

u(y) = u0 * ln(y/y0).

In this project, since the computational domain starts at y = 0 (the coastline), we set u(y) =u0 * ln(y+1) to ensure u = 0 at y = 0. To prevent unrealistic velocities far from shore, we apply a velocity cap:

u(y) = min [u0 ln(y+1), 0.03 m/s]

By comparing the movement of nitrate in this y-dependent current field with that in a uniform flow field of 0.03 m/s, we examine how a more realistic current field affects nitrate dispersion.

As mentioned above, we use the initial nitrate concentration as a Gaussian function for the baseline experiment, which implies that there is only a one-time release of nitrate in our time range. However, we can also consider the constant release of nitrate from rivers in the real world. Thus, we will implement continuous nitrate release as a boundary condition for comparison. In the constant release scenario, we set the initial nitrate concentration to zero in our spatial domain and modify the boundary condition to reflect constant nitrate discharge from the river as follows:

c(0,x,y) = 0

c(t,x,0) = x(t,x,dy) (if x =/= x0)

c(t,x0,0) = c(t,x0,0) + N_flux*dt

We calculated the total amount of nitrate at t=0 in the baseline scenario to ensure that the same amount of nitrate is injected over the entire time in the constant release scenario. Since the total nitrate in the baseline at t=0 was 14.20g, we set N_flux to 0.0142g/s so that the same amount of nitrate is injected over 1000 seconds.

AI reply:

In stage 2, we will conduct sensitivity experiments using our 2-dimensional numerical model. We will evaluate the model's performance by adjusting current velocity and diffusivity based on a baseline experiment. In addition, by applying a new flow field and nitrate modeling, we aim to gain insight into how nitrate moves in the real world.

The parameters of the sensitivity experiments are given by the following table:

For the baseline scenario, we will set a constant current velocity of 0.01~m/s, a diffusivity of 0.01~m^2/s, and the initial nitrate concentration as a Gaussian function. To investigate how advection and diffusion respond to flow velocity and diffusivity, we will use a current velocity of 1.0~m/s for the high velocity scenario and a diffusivity of 0.02~m^2/s for the high diffusion scenario.

For the flow field, we plan to implement a classic boundary layer flow, where the current velocity $u(y)$ follows a logarithmic profile relative to the distance from the shore (y) \cite{schlichting2016}:

\begin{equation}

u(y) = u_0 \ln(y/y_0).

\end{equation}

In this project, since the computational domain starts at $y = 0$ (the coastline), we set $u(y) = u_0 \ln(y+1)$ to ensure $u = 0$ at $y = 0$. To prevent unrealistic velocities far from shore, we apply a velocity cap:

\begin{equation}

u(y) = \min \left[u_0 \ln(y+1), 0.03~\text{m/s}\right]

\end{equation}

By comparing the movement of nitrate in this y-dependent current field with that in a uniform flow field of 0.03~m/s, we examine how a more realistic current field affects nitrate dispersion.

As mentioned above, we use the initial nitrate concentration as a Gaussian function for the baseline experiment, which implies that there is only a one-time release of nitrate in our time range. However, we can also consider the constant release of nitrate from rivers in the real world. Thus, we will implement continuous nitrate release as a boundary condition for comparison. In the constant release scenario, we set the initial nitrate concentration to zero in our spatial domain and modify the boundary condition to reflect constant nitrate discharge from the river as follows:

\begin{align}

c(0,x,y) &= 0 \\

c(t,x,0) &= c(t,x,\Delta y) \quad \text{(if } x \neq x_0\text{)} \\

c(t,x_0,0) &= c(t,x_0,0) + N_{\text{flux}} \cdot \Delta t

\end{align}

We calculated the total amount of nitrate at $t=0$ in the baseline scenario to ensure that the same amount of nitrate is injected over the entire time in the constant release scenario. Since the total nitrate in the baseline at $t=0$ was 14.20~g, we set N_{flux} to 0.0142~g/s so that the same amount of nitrate is injected over 1000 seconds.